LLM Training Memory Deep Dive

Why Training a 7B Model Requires ~4x More Memory Than Inference

A Complete Technical Reference: Parameters · Gradients · Optimizer States · Activations

---

THE CORE QUESTION

A 7B parameter model stored in FP16 requires ~14 GB for inference. Yet full training of the same model demands 58-112+ GB of GPU memory. Why the enormous gap?

1. Introduction

When you load a 7B parameter language model for inference, the GPU stores one thing: the weights. In FP16, that is roughly **14 GB**. But when you train the same model, that number balloons to **58 GB to 112+ GB**, depending on precision settings and batch size. This 4-8x multiplier is not a quirk — it is a structural necessity imposed by how neural network training works.

During training, the GPU must simultaneously hold four distinct categories of tensors:

(1) model **parameters**
(2) **gradients** of the loss with respect to each parameter
(3) **optimizer states** that accumulate gradient history and
(4) **activations** — intermediate values from the forward pass needed to compute gradients during backpropagation.

Each category has its own memory cost, precision requirements, and lifecycle.

These notes provide breakdown of every memory component.

Memory Component	Bytes per Param	7B Model	Applies To
FP16 Parameters	2 bytes	**14 GB**	Training + Inference
FP16 Gradients	2 bytes	**14 GB**	Training only
Adam Optimizer States (FP32)	8 bytes	**56 GB**	Training only
Activations (est. bs=1, s=2048)	~0.3 bytes	**~2 GB**	Training only
TOTAL (Mixed Precision Training)	~12-16 bytes	**~86-112 GB**	Training only

The numbers above follow from first principles, which this document derives step by step.

2. Memory Components in LLM Training

2.1 Model Parameters

What Are Parameters?

Parameters are the learnable tensors of a neural network — the weight matrices and bias vectors that define the model's learned function. In a transformer, these appear in:

- **Attention layers:** Query (W_Q), Key (W_K), Value (W_V), and output projection (W_O) matrices. Each has shape [hidden_dim × hidden_dim].
- **Feed-Forward layers:** Two linear projections per transformer block, typically expanding hidden_dim → 4×hidden_dim → hidden_dim.
- **Embeddings:** Token embedding table [vocab_size × hidden_dim] and positional encodings.
- **Layer norms:** Scale and shift parameters (two vectors of length hidden_dim per norm).

Memory Formula

For a model with **Φ** parameters stored at precision **p bytes/param**:

```
Memory_params = Φ × p

Where:
  Φ = number of parameters (e.g. 7,000,000,000 for a 7B model)
  p = bytes per parameter:
        FP32  → 4 bytes  (single precision, full range)
        FP16  → 2 bytes  (half precision, ±65504 range)
        BF16  → 2 bytes  (brain float, same exponent as FP32)
        INT8  → 1 byte   (quantized, used for inference/QLoRA)
        INT4  → 0.5 byte (quantized, used for QLoRA/GGUF)
```

Precision Comparison

Format	Bits	Bytes	7B Model Size	Notes
FP32	32	4	**28 GB**	Full precision. Used for optimizer states.
FP16	16	2	**14 GB**	Half precision. Fast on tensor cores. Overflow risk.
BF16	16	2	**14 GB**	Brain float. Preferred for training (wider range).
INT8	8	1	**7 GB**	Quantized inference. Not suitable for training.
INT4 / NF4	4	0.5	**3.5 GB**	QLoRA base model. Requires dequantization for compute.

Architecture Note: LLaMA-2 7B Parameter Breakdown

To ground the math in a real model, here is the approximate parameter breakdown for LLaMA-2 7B (hidden_dim=4096, 32 layers, 32 attention heads):

Component	Shape	Parameters	BF16 Memory

Token embedding	32000 × 4096	~131M	~0.26 GB
Attn Q, K, V (× 32 layers)	3 × (4096 × 4096) × 32	~1.61B	~3.22 GB
Attn output proj (× 32 layers)	(4096 × 4096) × 32	~537M	~1.07 GB
FFN gate + up + down (× 32)	3 × (4096 × 11008) × 32	~4.35B	~8.7 GB
Layer norms + misc	various	~small	~0.01 GB
TOTAL	—	**~6.74B***	**~13.5 GB**

**Rounded to 7B in marketing. Actual count varies slightly by config. All computations in this document use 7B = 7×10^9.*

2.2 Gradients

Role in Backpropagation

During the **forward pass**, data flows through the network and the model produces a prediction. During the **backward pass**, the chain rule of calculus propagates the loss gradient $\partial L/\partial W$ backward through every layer, computing the gradient of the loss with respect to every learnable parameter.

These gradients tell the optimizer which direction to move each parameter to reduce loss. Crucially, each gradient tensor has the same shape as its corresponding parameter tensor. If you have a weight matrix of shape [4096 × 4096], its gradient is also [4096 × 4096].

Memory Cost

Gradients are typically stored in the same precision as the parameters they belong to (FP16 or BF16 in mixed precision training):

```
Memory_gradients = Φ × p_grad

In mixed precision training:
  p_grad = 2 bytes (FP16 or BF16, matching the FP16 parameter copy)

For a 7B model:
  Memory_gradients = 7 × 10^9 × 2 bytes = 14 GB
```

> *Key insight: Gradients are NOT optional. You cannot skip storing them during the backward pass because you need them to update the parameters. This is 14 GB that inference never needs.*

Gradient Accumulation and Its Effect on Memory

Gradient accumulation is a technique where you process several micro-batches, accumulate their gradients, and only call the optimizer update after N micro-batches. This does NOT reduce gradient memory — gradients of the same shape must still be stored. It reduces the per-step batch size to fit in memory, at the cost of more forward/backward passes per optimizer step.

Why Gradients Must Exist Simultaneously with Parameters

During parameter update, you need both the current parameter values and their gradients simultaneously. You cannot discard parameters before computing the gradient update. This means the gradient buffer exists in GPU memory alongside the parameter buffer — not sequentially, but at the same time.

```
Peak memory during update step:
  Params (BF16)   +  Gradients (BF16)  =  2Φ + 2Φ = 4Φ bytes
                                       =  4 × 7B × 1 byte = 28 GB
(pre-optimizer)
```

2.3 Optimizer States (Adam / AdamW)

Why Optimizers Need Extra Memory

Simple gradient descent updates parameters by: $W \leftarrow W - \eta \cdot \nabla W$. This needs no extra state — only the current gradient. But plain SGD converges slowly and is sensitive to learning rates. Modern LLM training uses Adam (Adaptive Moment Estimation), which maintains a running history of gradients to produce smoother, more efficient updates.

Adam Optimizer: Mechanics

Adam maintains two exponential moving averages (EMAs) for every parameter:

- **First moment (m) — Momentum:** A smoothed version of the gradient. Reduces oscillations and accelerates convergence in the right direction.
- **Second moment (v) — Variance:** A smoothed version of the squared gradient. Automatically scales the learning rate per parameter based on how noisy its gradient is.

The update rule at each step t is:

```
Adam Update Rule:

  m_t = β₁ · m_{t-1}  +  (1 - β₁) · ∇W        [momentum update]
  v_t = β₂ · v_{t-1}  +  (1 - β₂) · (∇W)²      [variance update]

  m^_t = m_t / (1 - β₁^t)                        [bias correction]
  v^_t = v_t / (1 - β₂^t)                        [bias correction]

  W ← W - η · m^_t / (√v^_t + ε)                 [parameter update]

Typical defaults: β₁=0.9, β₂=0.999, ε=1e-8, η=3e-4
```

Memory Cost of Adam

Each of the two EMA buffers (m and v) is the same size as the parameter tensor. Crucially, for numerical stability, they are stored in FP32 even during mixed-precision (FP16/BF16) training:

```
Memory_optimizer (Adam, mixed precision):

  FP32 master params:   Φ × 4 bytes  =  4Φ  (for stable weight updates)
  FP32 first moment m:  Φ × 4 bytes  =  4Φ  (gradient EMA)
  FP32 second moment v: Φ × 4 bytes  =  4Φ  (squared gradient EMA)
                                        ____
  TOTAL:                12Φ bytes

For 7B model: 12 × 7 × 10^9 bytes = 84 GB

NOTE: Many implementations omit FP32 master params if already keeping
BF16 params:
  Without FP32 copy:   8Φ bytes = 56 GB for 7B (m + v only)
```

> *The optimizer states alone — before counting any activations — already require 4x more memory than the FP16 parameters. Adam is the single largest memory consumer in LLM training.*

AdamW vs Adam

AdamW (Adam with Weight Decay) applies weight decay directly to the parameters rather than to the gradients. Memory cost is **identical** to Adam — both maintain the same m and v buffers. AdamW

simply adds: $W \leftarrow W \times (1 - \eta \cdot \lambda)$ before applying the Adam gradient update, where λ is the weight decay coefficient. No additional memory.

Alternative Optimizers: Memory Comparison

Optimizer	Bytes/Param	7B Cost	Notes
SGD (no momentum)	0 bytes	0 GB	No state. Slow, rarely used for LLMs.
SGD + Momentum	4 bytes (FP32)	28 GB	One momentum buffer. Better than vanilla SGD.
Adam / AdamW	8 bytes (2×FP32)	**56 GB**	Standard for LLMs. m + v buffers. +FP32 params = 84GB.
8-bit Adam (bitsandbytes)	2 bytes (2×INT8)	14 GB	Quantized optimizer states. ~4x cheaper than Adam.
Adafactor	~4 bytes (factored)	~28 GB	Factorized second moment. Memory efficient.

2.4 Activations

What Are Activations?

Activations are the intermediate output tensors produced at every layer during the forward pass. When you pass a batch of tokens through a transformer, each layer computes and outputs a tensor that becomes the input to the next layer. All of these intermediate tensors must be retained in GPU memory during the forward pass so they can be used to compute gradients during backpropagation.

This is the fundamental challenge: the backward pass uses the chain rule, which requires knowing the activation values from the forward pass. You cannot compute $\partial L/\partial W$ for layer k without knowing the activations that were input to layer k.

Transformer Layer Activation Breakdown

For a single transformer block, the key intermediate tensors that must be stored include:

Tensor	Shape	Memory (BF16)
Layer norm input (2 per block)	2 × [b, s, h]	4sbh bytes
Q, K, V projections (shared input)	[b, s, h]	2sbh bytes
QK^T attention logits	[b, heads, s, s]	$2 \cdot a \cdot s^2 \cdot b$ bytes
Softmax output	[b, heads, s, s]	$2 \cdot a \cdot s^2 \cdot b$ bytes
Attention output (value vector)	[b, s, h]	2sbh bytes
Output projection	[b, s, h]	2sbh bytes
FFN up-projection output	[b, s, 4h]	8sbh bytes
FFN activation (GeLU/SiLU)	[b, s, 4h]	8sbh bytes
Dropout masks	[b, s, h]	sbh bytes (1 byte each)

Where: `b` = batch size, `s` = sequence length, `h` = hidden dimension, `a` = number of attention heads, `L` = number of layers.

Total Activation Memory Formula

Summing across all L transformer layers (from Megatron-LM's derivation):

```
Memory_activations (no recomputation) ≈ sbhL × (10 + 24/h·a·s) × 2
bytes

Simplified (with FlashAttention, which avoids O(s²) attention maps):
  Memory_activations ≈ 34 × s × b × h × L  bytes   (BF16)

For LLaMA-2 7B  (h=4096, L=32, s=2048, b=1):
  = 34 × 2048 × 1 × 4096 × 32 × 2 bytes
  = 34 × 2048 × 1 × 4096 × 32 × 2
  ≈ 18.3 GB   (without FlashAttention at b=1, s=2048)

With FlashAttention (avoids materializing attention scores):
  ≈ 2 × s × b × h × L = ~2.1 GB at b=1, s=2048
```

> *Activation memory scales LINEARLY with batch size and sequence length. At batch_size=32, s=2048, activations alone can exceed 60 GB — larger than all other components combined.*

Why Activations Are Not Needed for Inference

During inference, you only run the forward pass. The model produces an output token and the computation is complete. There is no backward pass, so there is no need to retain intermediate activation tensors. They can be discarded immediately after each layer is computed, keeping only the final output. This is why inference requires ~14 GB while training requires 60+ GB for the same model.

2.5 Temporary Buffers and Framework Overhead

Beyond the four main categories, real training workloads incur additional memory from several sources:

- **CUDA workspace buffers:** cuDNN and cuBLAS allocate internal scratch space for GEMM (matrix multiply) operations. For large matrix multiplications on A100/H100 GPUs, these buffers can be 256 MB–1 GB each.
- **PyTorch memory allocator fragmentation:** PyTorch uses a caching memory allocator that reserves GPU memory in blocks. Allocation/deallocation patterns create fragmented free space — you may have enough total free memory but be unable to allocate a single contiguous tensor. This typically adds 5–15% overhead.
- **Gradient communication buffers (DDP):** PyTorch's DistributedDataParallel (DDP) creates gradient bucket buffers per parameter group for all-reduce operations. These are roughly 4 bytes per parameter, adding ~28 GB for a 7B model.
- **Input data and tokenized batches:** The current batch of input token IDs, attention masks, and labels must reside on GPU. For batch_size=32, seq=2048, this is 32×2048×4 bytes ≈ 256 MB.
- **Loss and output logits:** The final logit tensor has shape [batch, seq, vocab_size]. For vocab_size=32000, batch=1, seq=2048: 1×2048×32000×2 bytes ≈ 131 MB.
- **Framework internals:** PyTorch autograd graph, Python objects, CUDA context, and driver overhead typically consume 0.5–2 GB.

In aggregate, temporary buffers and framework overhead typically add **5–20% on top of the calculated minimum memory**. Always budget 10–20% extra headroom over theoretical minimums when planning GPU allocations.

3. Complete Memory Breakdown: 7B Model

The following breakdown uses a LLaMA-2 7B model under standard mixed precision training conditions: BF16 parameters and gradients, FP32 optimizer states, batch_size=1, sequence_length=2048.

Component	Bytes/Param	Formula	Memory	Explanation
BF16 Parameters	2	2Φ	**14 GB**	Model weights in BF16
BF16 Gradients	2	2Φ	**14 GB**	∂L/∂W per parameter
FP32 Momentum (m)	4	4Φ	**28 GB**	Adam first moment EMA
FP32 Variance (v)	4	4Φ	**28 GB**	Adam second moment EMA
Activations (bs=1, s=2048)	~0.3	34sbhL (bytes)	**~2 GB**	Forward pass intermediates
Framework / CUDA overhead	—	~fixed	~2 GB	PyTorch, cuDNN workspace
TOTAL (Training)	~12 bytes	~12Φ + overhead	**~88 GB**	Full mixed precision pipeline

Understanding the 4× Multiplier

The '4× model size' figure is computed by comparing inference memory to training memory at the same base precision:

```
Inference memory  =  2Φ bytes  (BF16 weights only)
                  =  14 GB for 7B model

Minimum training memory (no FP32 master copy):
  = BF16 params + BF16 grads + FP32 m + FP32 v
  = 2Φ + 2Φ + 4Φ + 4Φ
  = 12Φ = 84 GB
  Ratio: 12Φ / 2Φ = 6× inference

Vs. FP32 model baseline (Φ×4 = 28 GB):
  Training (16Φ with FP32 master copy) = 112 GB
  Ratio: 112 / 28 = exactly 4×  ← origin of the '4× rule'

GaLore paper measured: 14GB params + 28GB grads + 14GB states + 2GB
acts ≈ 58GB
(Using BF16, single batch, no FP32 master copy)
```

The exact multiplier depends on training configuration, but the **3×–8× range** relative to inference is typical. The '4×' figure is a useful rule of thumb that assumes FP32 parameters as the baseline.

4. Training vs. Inference Memory

The memory gap between training and inference is not a matter of implementation choices but of mathematical necessity. Here is a direct comparison:

Memory Component	Inference	Training	Reason for Difference
Parameters	14 GB (BF16)	14–42 GB	Training may keep FP32 master copy for stability
Gradients	None — 0 GB	14 GB	No backward pass in inference
Optimizer states	None — 0 GB	28–84 GB	No parameter updates in inference
Activations	Minimal (discarded)	2–60+ GB	Must be retained for backward pass
KV Cache (inference only)	2–20+ GB	None	Caches K/V for each token at generation time
TOTAL (7B, bs=1)	**~14–16 GB**	**~58–112 GB**	Training = 4–8× inference memory

The Forward-Backward Pass Memory Timeline

Understanding when memory is allocated and freed is critical for budgeting:

- **t=0 (Model load):** Parameters loaded into GPU. 14 GB allocated.
- **t=1 (Forward pass begins):** Input batch placed on GPU. Activations accumulate layer by layer. Peak activation memory reached at end of forward pass.
- **t=2 (Forward pass ends):** Loss computed. All L×(activation tensors) still live in memory. Parameters still needed. Total: 14+2 GB = ~16 GB.
- **t=3 (Backward pass begins):** Gradient buffers allocated. 14 GB more. Activations consumed and freed layer by layer in reverse order. Total peak: ~30 GB.
- **t=4 (Optimizer step):** Optimizer state tensors accessed. All four components live simultaneously: params + grads + m + v = 84 GB peak.
- **t=5 (After optimizer step):** Gradients zeroed or freed. Memory drops to ~42 GB (params + optimizer states). Cycle restarts.

> *The true peak memory occurs at the optimizer step, not during the forward or backward pass. Systems that only measure forward-pass memory will underestimate GPU requirements by 3-5×.*

5. Techniques to Reduce Training Memory

When GPU memory is insufficient for full training, engineers can apply a hierarchy of techniques, each with different trade-offs between memory savings, training speed, and model quality.

5.1 Mixed Precision Training (BF16/FP16 + FP32)

Mixed precision training is the standard baseline for LLM training. The key idea: use FP16/BF16 for the forward and backward pass (fast tensor core operations), but maintain FP32 master weights for the parameter update step (prevents precision loss accumulation).

```
Mixed Precision Memory (ZeRO paper formulation):
  FP16 params:          2Φ bytes
  FP16 gradients:       2Φ bytes
  FP32 master params:   4Φ bytes
  FP32 momentum m:      4Φ bytes
  FP32 variance v:      4Φ bytes
  ──────────────────────────────
  Total:               16Φ bytes  =  112 GB for 7B

Memory saving vs. pure FP32 training (20Φ bytes):
  Saves: 4Φ bytes (the FP32 gradient buffer replaced by FP16)
```

BF16 is preferred over FP16 for modern training because it has the same exponent range as FP32 (preventing overflow/underflow), while FP16 can overflow easily with large gradients.

5.2 Gradient Checkpointing (Activation Recomputation)

Gradient checkpointing trades compute for memory by discarding most activations during the forward pass and recomputing them on-demand during the backward pass. Only checkpoint activations at selected layer boundaries are stored.

```
Memory_activations (full recomputation) = 2 × s × b × h × L  bytes
  vs. no recomputation: ~34 × s × b × h × L  bytes

Memory savings: ~17× reduction in activation memory
Compute cost:   ~33% more FLOPs (one extra forward pass per batch)

For 7B model (s=2048, b=1, h=4096, L=32):
  With checkpointing:   ~2.1 GB  activations
  Without checkpointing: ~18 GB  activations
```

`torch.utils.checkpoint.checkpoint()` is the PyTorch API. In HuggingFace: `model.gradient_checkpointing_enable()`

5.3 ZeRO Optimizer (Zero Redundancy Optimizer)

ZeRO (from DeepSpeed) partitions optimizer states, gradients, and parameters across multiple GPUs in a data-parallel setup, eliminating the redundancy where every GPU stores the full model state.

ZeRO Stage	What Is Partitioned	Memory/GPU (N GPUs)	Communication Cost
Stage 1	Optimizer states	12Φ / N + 4Φ	Low — standard AllReduce
Stage 2	Optimizer states + Gradients	2Φ + 14Φ / N	Moderate — ReduceScatter

Stage 3	Optimizer states + Grads + Parameters	16Φ / N	High — AllGather per forward layer

ZeRO Stage 3 enables training models that are 10× larger than a single GPU can hold, at the cost of significantly higher interconnect traffic. For LLaMA-2 7B on 8× A100 80GB GPUs, ZeRO Stage 3 reduces per-GPU memory to ~14 GB.

5.4 FSDP (Fully Sharded Data Parallel)

PyTorch's native equivalent of ZeRO Stage 3. FSDP shards parameters, gradients, and optimizer states across the data-parallel group. It gathers parameters just-in-time for each layer's forward/backward pass, then immediately discards the gathered copy. Memory per GPU approaches 16Φ / N.

5.5 LoRA / PEFT (Parameter-Efficient Fine-Tuning)

LoRA freezes the pre-trained weights and injects small trainable rank-decomposition matrices into selected layers. Instead of training all Φ = 7B parameters, you train only ~0.1–1% of them.

```
LoRA memory vs. full fine-tuning:

  Trainable params (Φ_lora) ≈ 0.1% × 7B = 7M parameters
  Frozen base model (BF16): 14 GB  (no gradients needed)

  Gradient memory:  2 × 7M bytes   = 14 MB    (vs 14 GB)
  Optimizer states: 8 × 7M bytes   = 56 MB    (vs 56 GB)

  TOTAL fine-tuning memory: ~14.1 GB  (vs 88 GB full training)
  Saving: ~6× memory reduction
```

Trade-off: LoRA can underperform full fine-tuning for large distribution shifts. For most domain adaptation tasks, quality is comparable to full fine-tuning with a fraction of the memory.

5.6 Quantization (QLoRA and GPTQ)

QLoRA (Quantized LoRA) takes LoRA further by storing the frozen base model in 4-bit NF4 quantization, reducing it from 14 GB to ~3.5 GB. LoRA adapters remain in BF16. During computation, weights are dequantized on-the-fly to BF16 for matrix multiplications.

```
QLoRA memory estimate for 7B model:
  4-bit base model:          3.5 GB
  BF16 LoRA adapters + grads: ~0.1 GB
  FP32 optimizer states:     ~0.1 GB
  Activations:               ~2 GB
  ──────────────────────────────────
  TOTAL:                   ~5.7 GB  ← fits on a single 8GB GPU!
```

5.7 CPU Offloading

Optimizer states (the largest component) can be offloaded to CPU RAM and brought back to GPU only during the optimizer step. DeepSpeed ZeRO-Offload and ZeRO-Infinity implement this. CPU RAM is typically 8–16× larger than GPU VRAM but 100× slower.

Trade-off: For most training workloads, offloading optimizer states reduces training throughput by 20–50% depending on PCIe bandwidth. It is most useful for fine-tuning when GPU memory is a hard constraint.

6. Visual Illustrations

6.1 Training Memory Pipeline (Per Training Step)

The following diagram shows how GPU memory accumulates and is released across a single training step:

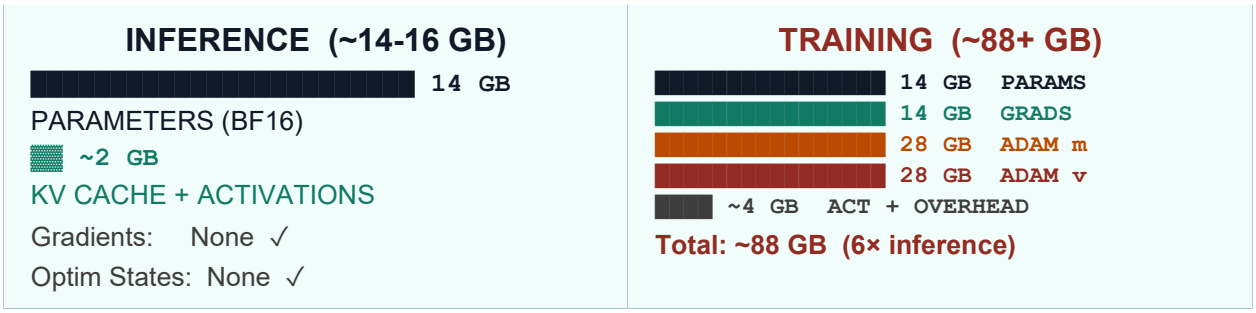
GPU MEMORY TIMELINE — ONE TRAINING STEP (7B MODEL, BF16)

Phase:	Params	Grads	Optim States	Activations	TOTAL
Model load	14 GB	-	56 GB	-	70 GB
Forward pass	14 GB	-	56 GB	0→18 GB	88 GB
Loss computed	14 GB	-	56 GB	18 GB	88 GB
Backward pass	14 GB	0→14 GB	56 GB	18→0 GB	~88 GB
Optimizer step PEAK	**14 GB**	**14 GB**	**56 GB**	**0**	**84 GB** ←
Grads zeroed	14 GB	-	56 GB	0	70 GB

NOTE: Activations grow layer-by-layer and shrink during backward.
Gradient checkpointing reduces activation peak from 18GB to ~2GB.

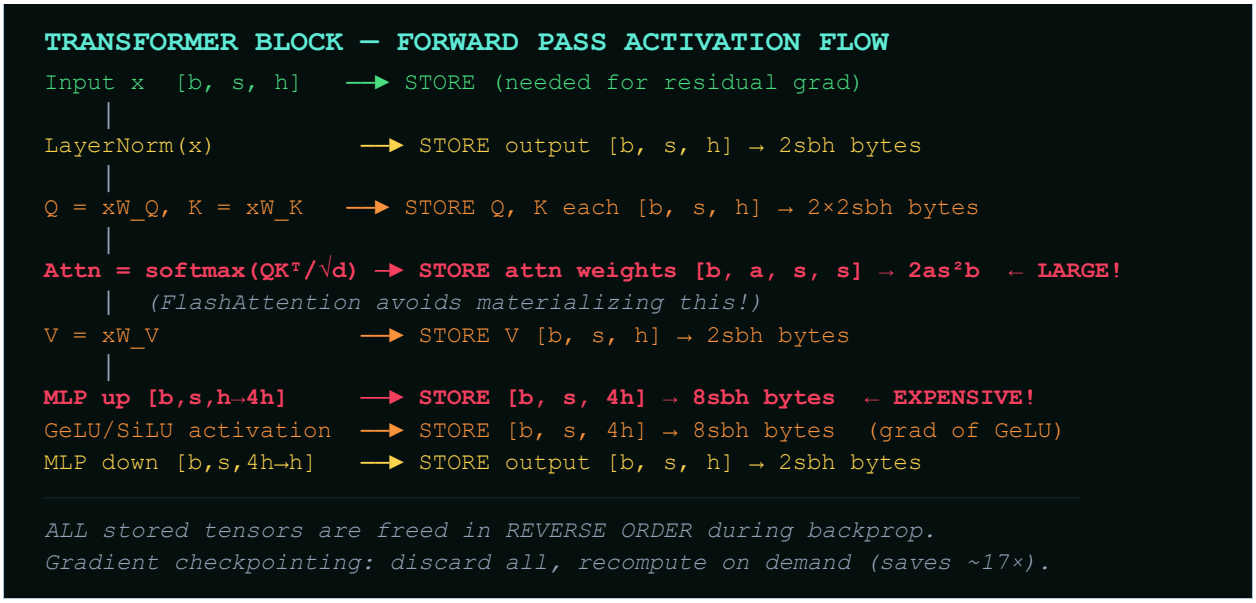
6.2 Memory Composition: Inference vs. Training

A proportional view of where memory goes in each scenario:



6.3 Activation Memory Flow Through a Transformer Layer

The following shows the order of tensor creation within one transformer block during the forward pass, and which tensors must survive until the backward pass:

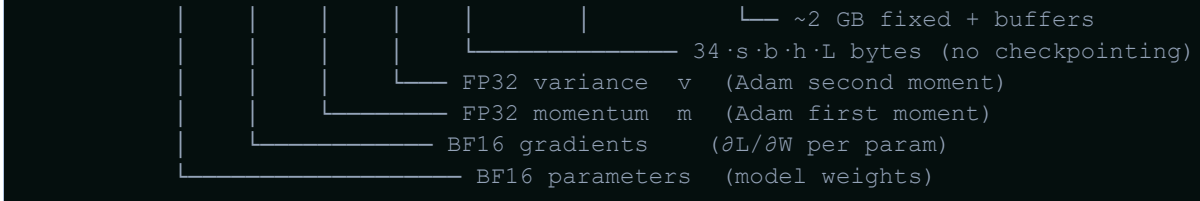


7. Key Takeaways

1. **Why training memory is large**
   Training requires four simultaneous memory allocations: parameters, gradients, optimizer states, and activations. Each has its own precision requirement and lifecycle. None can be eliminated — they are each structurally necessary for gradient-based learning.
2. **The optimizer is the biggest consumer**
   Adam's two FP32 buffers (m and v) each require 4 bytes per parameter — twice the size of the BF16 parameter tensor itself. For a 7B model, optimizer states alone consume 28-56 GB. This is why the 4× rule of thumb exists.
3. **Activations scale with batch × sequence length**
   Unlike parameters (fixed), activations grow linearly with batch size and quadratically with sequence length (without FlashAttention). A batch of 32 with 2048 tokens can produce 60+ GB of activations — far exceeding all other components combined.
4. **Inference saves memory by eliminating three components**
   Inference removes gradients (no backward pass), optimizer states (no parameter updates), and most activations (no need to retain for backprop). Only parameters and a minimal KV cache are needed.
5. **The 4× rule assumes FP32 baseline**
   Training in full mixed precision (16Φ bytes) vs. FP32 inference (4Φ bytes) gives exactly 4×. But relative to FP16 inference (2Φ bytes), training is 6-8×. The multiplier depends on which baseline you use.
6. **Gradient checkpointing is the best first optimization**
   Enabling gradient checkpointing costs only ~33% extra compute but reduces activation memory by 17×. For most models, this single technique prevents the largest source of OOM errors at minimal quality cost.
7. **ZeRO / FSDP enables training larger-than-memory models**
   By sharding all four memory components across N GPUs, ZeRO Stage 3 reduces per-GPU memory to 16Φ/N. On 8× A100 80GB GPUs, a 70B model becomes trainable. This is the standard approach for production LLM training.
8. **LoRA and QLoRA are transformative for fine-tuning**
   LoRA reduces trainable parameters by 99%, shrinking gradients and optimizer states proportionally while keeping the frozen base model resident. QLoRA quantizes the base model to 4-bit, bringing 7B fine-tuning to a single consumer GPU.

MASTER FORMULA: Full Mixed Precision Training Memory

$$M_{total} = 2\Phi + 2\Phi + 4\Phi + 4\Phi + 4\Phi + M_{act} + M_{overhead}$$



For 7B model: 14 + 14 + 28 + 28 + 28 + ~2 + ~2 = ~116 GB

Abhyuday Patel